

Controls / Optimization HW 2

Sina Hazeghi

September 12, 2026

1 2D-Quadrotor

For some positive constants a, I, m we have system dynamics:

$$q = \begin{bmatrix} y \\ z \\ \theta \end{bmatrix}, \quad x = \begin{bmatrix} q \\ \dot{q} \end{bmatrix}, \quad \ddot{q}(q, u) = \begin{bmatrix} -\frac{\sin \theta}{m}(u_1 + u_2) \\ -g + \frac{\cos \theta}{m}(u_1 + u_2) \\ \frac{a}{I}(u_1 - u_2) \end{bmatrix}$$

Writing in terms of state $x \in \mathbb{R}^6$:

$$x = \begin{bmatrix} q \\ \dot{q} \end{bmatrix}, \quad \dot{x} = f(x, u) = \begin{bmatrix} \dot{q} \\ \ddot{q} \end{bmatrix} = \begin{bmatrix} \dot{y} \\ \dot{z} \\ \dot{\theta} \\ -\frac{\sin \theta}{m}(u_1 + u_2) \\ -g + \frac{\cos \theta}{m}(u_1 + u_2) \\ \frac{a}{I}(u_1 - u_2) \end{bmatrix}$$

We define cost as:

$$\mathcal{J} = \int_0^\infty x(t)^T Q x(t) dt$$

with $Q \succ 0, Q = Q^T$

- We can see that dynamics are linear for constant θ and $\ddot{\theta} = 0$ iff $u_1 = u_2$
- For fixed points where $f(x^*, u^*) = 0$ and we linearize this nonlinear system will assume linearized (A, B) stabilizable s.t. Ricatti Equation has unique solution for S and we can create approx. Optimal LQR controller

a) Defining Q

For each find Q that works or explain why DNE:

i) Stabilize at $q=0$:

Need some fixed point (x^*, u^*) s.t. $f(x^*, u^*) = 0$ and we can linearize around this point:

$$\ddot{q}(0, u) = \begin{bmatrix} 0 \\ -g + \frac{1}{m}(u_1 + u_2) \\ \frac{a}{I}(u_1 - u_2) \end{bmatrix} \stackrel{\text{set}}{=} 0$$

So we see that $u^* = \frac{mg}{2} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$. Since $R = 0$, we are ok with having nonzero control over infinite time horizon.

We can pick $Q = I$: satisfies PSD & Symmetric. Furthermore once we reach state

$$x(t) = \begin{bmatrix} q \\ \dot{q} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Then $x(t)^T Q x(t) = 0$ so the cost will converge

ii) Drive the quadrotor to line $2y=z$:

Since origin $x^* = 0$ is on this line we know $\exists u^*$ s.t. both constraint and $f(x^*, u^*) = 0$ is satisfied so we can linearize about this fixed point for our LQR controller.

Now we need to design Q s.t. $x(t)^T Q x(t) = 0$ if $z = 2y$. We can design Q in terms of smaller matrices:

$$Q = \begin{bmatrix} M & 0 \\ 0 & I \end{bmatrix}$$

Where M s.t. $Mq = 0$ when $q[0 : 2] = [y, z]^T = c[1, 2]^T$ regardless of θ . We can accomplish this via dot product

$$m^T q; m = (2, -1, 0)^T$$

And we see that the function $q^T m m^T q \geq 0$ and is 0 for q on the line $y = 2z$, but $\neq 0$ otherwise so we just choose

$$M = m m^T = \begin{bmatrix} 4 & -2 & 0 \\ -2 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Notice it's really 2x2 matrix embedded in the 3x3 because we are not assigning any cost to θ so long as our position is on the line. Thus we have an effective Q for our objective

iii) Drive to the curve $z = y^2$

Since $z = y^2$ is a nonlinear constraint, $\nexists Q$ s.t. it will only assign 0 cost when we are on this curve globally.

With the quadratic cost form $x^T Q x$ we can only assign 0 cost to linear constraints by making the constraints part of the null space of Q

b) Scaling Q

Suppose for $Q = Q_1$ we have optimal policy and cost-to-go: u_1^* and V_1^* . For $Q = Q_2 = \frac{1}{2}Q_1$ we have u_2^* and V_2^* . What's the relationship between the 2 optimal policies and cost-to-go?

Consider the HJB in this form:

$$V^*(x(t)) = \min_u V^*(x(t) + \Delta t f(x, u)) + Q(x)$$

where $Q(x) = x(t)^T Q x(t)$ here but could be any generic cost formulation defined via Q . Then notice we can multiply both sides by any constant c and the equality remains so we have that:

$$cQ \propto cV$$

by multiplying our current cost by some c , our cost-to-go is scaled by the same amount. We can also see that in the formulation of V as a sum of all costs:

$$V = \int Q(x) dt \implies cV = \int cQ(x) dt$$

Now to consider how our optimal policy u should change given our cost-to-go has been scaled: u^* is optimal control that minimizes $V^*(x(t) + \Delta t f(x, u))$, notice that for any function f , scaling it by some constant $c > 0$, cf it's critical points (where $f' = 0$) remain the same so as long as $c > 0$ $\min u$ would be the same.

So to answer the homework question the following is true when Q is scaled by $\frac{1}{2}$:

- $V_2^* = \frac{1}{2} V_1^*$
- $u_1^* = u_2^*$

2 Trajectory Tracking with Time Varying LQR

Consider our 2D Quadrotor system. Let $x_d(t)$ be our desired trajectory and $u_d(t)$ be our input controls to follow it.

We define our trajectory error $x_e(t)$ as simply the distance we currently are from our desired trajectory:

$$x_e(t) = x(t) - x_d(t)$$

and we apply *Time-Varying LQR* to derive a controller to achieve stable equilibria $x_e = 0$ via applying error-correcting control to get final control law:

$$u(x_e, t) = u_e(x_e, t) + u_d(t)$$

a) Linearizing our Dynamics at (x_d, u_d)

To apply LQR controller we linearize where $x_e = 0$, namely at $x(t) = x_d(t)$, and notice that we get $u(0, t) = u_d(t)$ at this point. So we can approximate our error dynamics:

$$\dot{x}_e = \dot{x} - \dot{x}_d = f(x(t), u(t)) - f(x_d(t), u_d(t))$$

Applying first order Taylor expansion at $(x, u) = (x_d, u_d)$:

$$\dot{x}_e(t) \approx \frac{\partial f(x_d, u_d)}{\partial x} x_e + \frac{\partial f(x_d, u_d)}{\partial u} u_e$$

Recall in part 1 we defined f:

$$x = \begin{bmatrix} q \\ \dot{q} \end{bmatrix}, \quad \dot{x} = f(x, u) = \begin{bmatrix} \dot{q} \\ \ddot{q} \end{bmatrix} = \begin{bmatrix} \dot{q} \\ \ddot{q} \end{bmatrix} = \begin{bmatrix} \dot{q} \\ -\frac{\sin \theta}{m}(u_1 + u_2) \\ -g + \frac{\cos \theta}{m}(u_1 + u_2) \\ \frac{a}{I}(u_1 - u_2) \end{bmatrix}$$

We compute the Jacobians:

$$\frac{\partial f}{\partial x} = \begin{bmatrix} \frac{\partial \dot{q}}{\partial q} & \frac{\partial \ddot{q}}{\partial \dot{q}} \end{bmatrix} = \begin{bmatrix} 0_3 & I_3 \\ M_{\ddot{q}, q} & 0_3 \end{bmatrix} = A(t)$$

Where

$$M_{\ddot{q}, q} = \begin{bmatrix} 0 & 0 & -\frac{\cos \theta_d(t)}{m}(u_1, d(t) + u_2, d(t)) \\ 0 & 0 & -\frac{\sin \theta_d(t)}{m}(u_1, d(t) + u_2, d(t)) \\ 0 & 0 & 0 \end{bmatrix}$$

And the Jacobian of f w.r.t. to our controls u is:

$$\frac{\partial f}{\partial u} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ -\frac{\sin \theta_d(t)}{m} & -\frac{\sin \theta_d(t)}{m} \\ \frac{\cos \theta_d(t)}{m} & \frac{\cos \theta_d(t)}{m} \\ \frac{a}{I} & -\frac{a}{I} \end{bmatrix} = B(t)$$

Using this linearization we have our locally linear error dynamics:

$$\dot{x}_e(t) \approx A(t)x_e + B(t)u_e$$

b) Deriving Time-Varying LQR

We define our cost:

$$g(t, x_e, u_e) = x_e(t)^T Q x_e(t) + u_e(t)^T R u_e(t)$$

And thus we have objective function which tells us our score given any error trajectory / control inputs from $[0, t_f]$:

$$\mathcal{J} = \frac{1}{2} \int_0^{t_f} g(t, x_e, u_e) dt + \frac{1}{2} x_e(t_f)^T Q_f x_e(t_f)$$

We guess that our time-varying cost-to-go function V^* is quadratic of the form:

$$V^*(t, x_e) = \frac{1}{2} x_e(t)^T S(t) x_e(t)$$

Given this formulation derive the optimal policy and some formulation of S (time-varying ricatti equation).

Starting with the HJB:

$$V^*(t, x_e) = \min_{u_e} [g(t, x_e, u_e) dt + V^*(t + dt, x_e + f(t, x_e, u_e) dt)]$$

We apply Taylor approximation:

$$V^*(t + dt, x_e + f dt) \approx V^*(t, x_e) + \frac{\partial V^*}{\partial t} dt + \frac{\partial V^*}{\partial x} f dt$$

We plug this in, divide by dt , and move $\frac{\partial V^*}{\partial t}$ outside of the min since it doesn't depend on u :

$$-\frac{\partial V^*}{\partial t} = \min_{u_e} \left[\frac{\partial V^*}{\partial x} f + g(t, x_e, u_e) \right]$$

Now we apply the fact that $V^*(t, x_e) = x_e(t)^T S(t) x_e(t)$ so we know:

- $\frac{\partial V^*}{\partial t} = \frac{1}{2} x_e^T \dot{S}(t) x_e$
- $\frac{\partial V^*}{\partial x} = x_e^T S(t)$

To take the min of u_e then we must solve:

$$\frac{\partial}{\partial u} (x_e^T S(t) f + g(t, x_e, u_e)) = 0$$

Using the fact that $\frac{\partial f}{\partial u} = B(t)$ from above this simplifies to:

$$u_e^T R = -x_e^T S(t) B(t)$$

We take transpose of both sides and apply R^{-1} to get optimal control policy ($S(t)$ is symmetric):

$$u_e^*(t, x_e) = -R^{-1} B(t)^T S(t) x_e$$

Plugging this back into HJB to remove the min:

$$-\frac{\partial V^*}{\partial t} = \frac{\partial V^*}{\partial x} f(t, x_e, u_e^*) + g(t, x_e, u_e^*)$$

With $f = A(t)x_e + B(t)u_e$:

$$-\frac{1}{2} x_e^T \dot{S} x_e = x_e^T S (A x_e + B u_e^*) + \frac{1}{2} x_e^T Q x_e + \frac{1}{2} u_e^{*T} R u_e^*$$

After plugging in form for u_e^* and applying $x_e^T S A x_e = \frac{1}{2} x_e^T (S A + A^T S) x_e$ and then factoring out the x_e we finally get the time-varying Ricatti Equation:

$$-\dot{S}(t) = S(t) A(t) + A(t)^T S(t) - S(t) B(t) R^{-1} B(t)^T S(t) + Q$$

With boundary condition that $S(T) = Q_f$ so we can integrate backwards in time to solve the ODE

c) Solving the Time-Varying Ricatti Equation

Typically instead of trying to solve numerically via integrating backward from Q_f it's more numerically stable to factor $S(t) = L(t) L(t)^T$

Where we have the ODE for L :

$$\dot{L}(t) = -\frac{1}{2} Q L(t)^{-T} - A(t)^T L(t) + \frac{1}{2} L(t) L(t)^T B(t) R^{-1} B(t)^T L(t)$$

i) Confirming $S = LL^T$ factorization

Note: For this section I will be omitting the time-varying notation for simplicity, but for context keep in mind that certain matrices are functions of time t

To ensure our factorization is correct we must use the ODE in L to show that $\dot{S} = \frac{d}{dt}LL^T$

$$\frac{d}{dt}LL^T = \dot{L}L^T + L\dot{L}^T$$

We compute each term:

$$\dot{L}L^T = -\frac{1}{2}QL^{-T}L^T - A^TLL^T + \frac{1}{2}LL^TBR^{-1}B^TLL^T = -\frac{1}{2}Q - A^TS + \frac{1}{2}SBR^{-1}B^TS$$

$$L\dot{L}^T = -\frac{1}{2}LL^{-1}Q - LL^TA + \frac{1}{2}LL^TBR^{-1}B^TLL^T = -\frac{1}{2}Q - SA + \frac{1}{2}SBR^{-1}B^TS$$

Summing these 2 terms together we can confirm we get back the time-varying Ricatti Equation:

$$\dot{S} = \dot{L}L^T + L\dot{L}^T = -Q - SA - A^TS + SBR^{-1}B^TS$$

So we see that factorization of S into L , where L has that particular ODE is valid

d) Applying Time-Varying LQR to Stabilize 2D Quadrotor

We apply what we've learned to derive a time-varying LQR controller that enables a 2D quadrotor to follow a desired trajectory despite trajectory error (initial state $\neq$ trajectory initial state).

This part will give you an overview of how the quadrotor simulation works (see code here). `trajectory.py` and `create_animation.py` are self-explanatory so we omit them.

`quad_sim.py`

In this file we simply write the nonlinear equations of motion $\dot{x} = f(x, u)$ (using some derived constants for g, m, a, I) and then apply them with `simulate_quadrotor(x0, tf, quadrotor)` which integrates forward in time given initial state x_0 , and controller `quadrotor`, until time `tf`

`quadrotor.py`

Given our cost weight matrices: Q, R, Q_f and our desired trajectory / inputs $x_d(t), u_d(t)$ we write our stabilizing TV-LQR controller in the following manner:

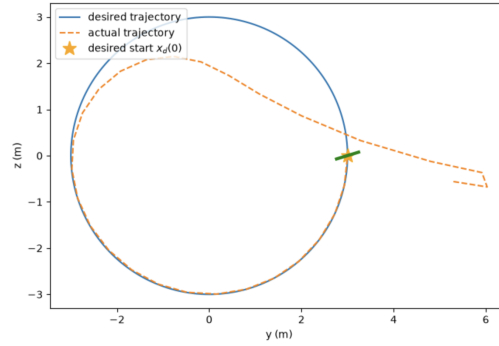
- Compute linearized dynamics $\dot{x}_e(t) = A(t)x_e + B(t)u_e$ where $A(t), B(t)$ are the Jacobians of nonlinear dynamics w.r.t. desired state x_d and precomputed controls u_d we derived in part a)
- We decompose $S(t_f) = Q_f$ into lower triangular matrices $Q_f = L(t_f)L(t_f)^T$ via Cholesky Decomposition.
- $L(t_f)$ is the "final" matrix we need to compute $V^*(t_f, x) = x^TS(t_f)x$. To compute the cost-to-go at any time $t \in [0, t_f]$ we take our ODE for L from part c), integrate backwards in time from $t_f \rightarrow 0$, and interpolate.

- This is all accomplished via `solve_ivp(_ldot, [tf. 0], lf, dense_output=True)`. Passing in the inverted timespan tells `solve_ivp` to integrate backwards, and using `dense_output=True` tells `scipy` to keep and stitch together the Hermitian Polynomials used by the Runge-Kutta solver to integrate so we can also use these for interpolation
- Now that we can compute $L(t)$ at any time we can easily apply our optimal feedback control law $u_e = -R^{-1}B(t)L(t)L(t)^T$ and add this to $u_d(t)$ to get our stabilizing, trajectory-tracking LQR controller

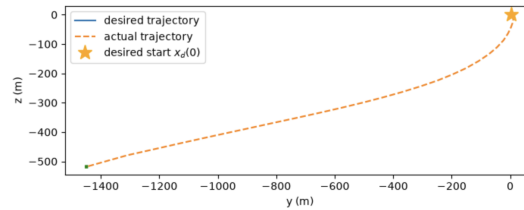
It's surprisingly robust, still tracking trajectories even with initial $\|x_e(0)\| = 10$. Given a starting state far from our desired $x_d(0) = (3, 0, 0.3, 0, 3, 0.0855)^T$

$$x(0) = \begin{bmatrix} 5.29746474 \\ -0.56047902 \\ 0.30048827 \\ 4.36064589 \\ -3.03756859 \\ -5.49759035 \end{bmatrix}$$

We get the converging trajectory:



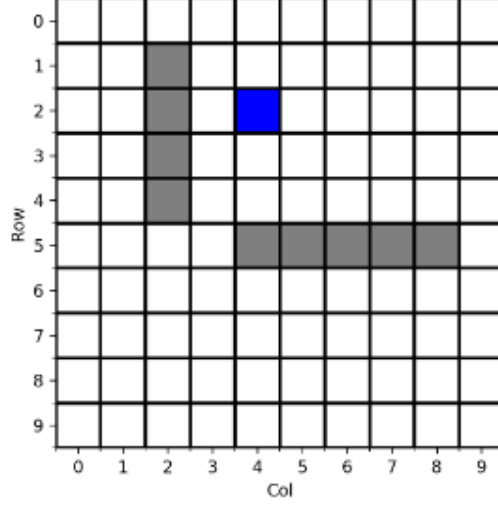
However, with this initial state of $x(0) = [4.11852286, -1.75788021, -3.29855026, 5.06005968, -6.72514113, -0.574]$ our trajectory completely diverges from what we expected:



And this is because LQR is computing the optimal stabilizing feedback based on dynamics linearized at $x_d(t), u_d(t)$. Once our actual state gets far enough from this LQR's assumptions break (since linearization only gives us local information about the dynamics) and the nonlinear nature of our system results in x_e compounding and thus making our controller less and less accurate (and also resulting in larger u_e).

3 Grid World Value Iteration

We implement a grid world with possible actions up, right, down, left where we are trying to reach the goal state as seen in blue in this figure:



Actions are stochastic with a slip probability that determines if action actually moves us to a new state or not. We have costs = 1 (base cost for any action), 2 (cost for hitting grid world walls), and 5 (cost for hitting any of the gray "sticky" states).

We implement a P transition matrix where $P(s, a)$ gives us a list of next possible states s' , their transition probs $p(s'|s, a)$ and cost $g(s, a)$.

Once we have this we implement the Value Iteration algorithm:

Algorithm 1 Value Iteration

```

 $k \leftarrow 1$ 
Initialize  $V_0(s) \leftarrow 0$  for all  $s$ 
while not converged do
  for each state  $s$  do
     $V_k(s) \leftarrow \min_a \sum_{s'} P(s'|s, a)(g(s, a, s') + \gamma V_{k-1}^*(s'))$ 
  end for
   $k \leftarrow k + 1$ 
end while
return  $V$ 

```

- Notice we replaced $\min_a E_{s'}[g + \gamma V_{k-1}^*(s')]$ with a sum over discrete probabilities
- In practice we check convergence via $\text{converged} = (\max_s V_k - V_{k-1} < \text{tol})$

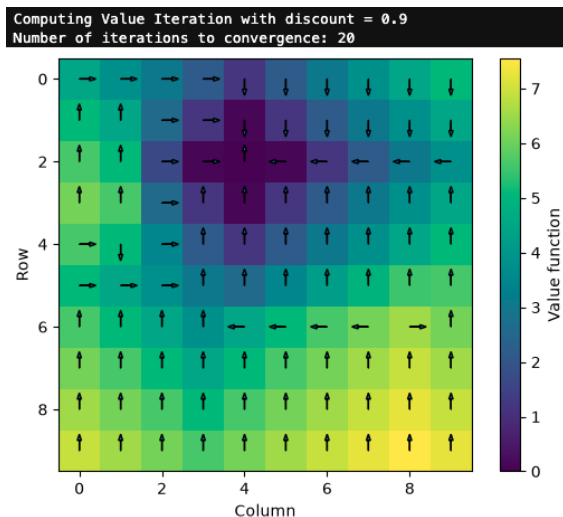
Once we have V^* computing u^* for discrete actions is trivially:

$$u^*(s) \leftarrow \arg \min_a E_{s'}[g + \gamma V_{k-1}^*(s')]$$

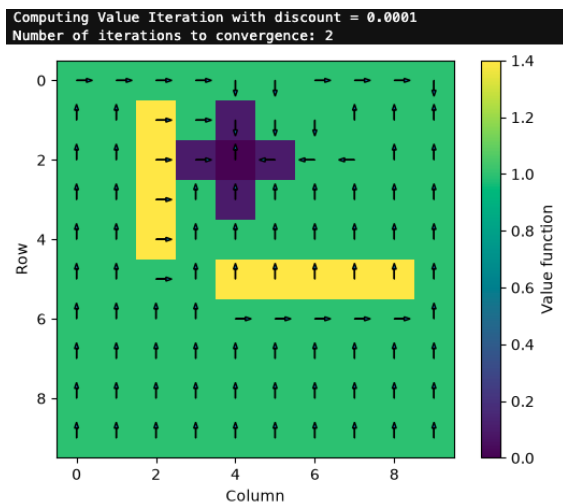
c) Discount Factor Analysis

We run value iteration for multiple $\gamma \in (0,1)$ and notice that for smaller γ it converges much faster. Intuitively this makes sense since as $\gamma \rightarrow 0$ our value iteration problem is basically $V(s) \rightarrow E_{s'}[g(s',a)]$ and our policy becomes more greedy to the point that it doesn't have enough information to even reach the goal unless it's in an adjacent state to the goal and thus gives 0 cost.

This can be seen when comparing our value iteration visualizations



Above figure is for $\gamma = 0.9$



Above figure is for $\gamma = 10^{-6}$

As you can see when comparing the 2nd figure to the 1st, with smaller gamma the policy is acting much more greedily, even when its super close to the goal it's moving away from it (an artifact of tie breaker preference when choosing min action). When it comes close to the "sticky" states it goes all the way around it (even though going through would make more sense since it would be faster to reach the goal). We also see that it converges an order of magnitude faster when $\gamma \approx 0$ and that's because it no longer really needs to aggregate any future information to formulate V^* and u^* before it reaches our convergence criteria